Course: AI Champions Programme

GitHub Copilot Enterprise, Spec-Driven Development, MCP & Agentic Software Engineering

NIIT StackRoute Programme Design | Honeywell Software Engineering Context

Revised Edition — Module-wise design updated per delivered-batch observations and stakeholder feedback | Status: Draft for Sign-off

© NIIT, All information within this document is intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Program Introduction

The programme follows an incremental engineering journey rather than a collection of AI tool topics. Participants begin with a mandatory self-paced GenAI/LLM foundations pre-read, then use Module 1 to recap course objectives and core concepts together as a group. They then build advanced GitHub Copilot capability, recap prompt engineering and move into context engineering, learn Spec-Driven Development (SDD) using GitHub Spec Kit, practise a complete greenfield SDLC, learn brownfield evolution and app modernization, validate SDD-developed features through a mock/stub-based test strategy, extend agents with MCP — including building a custom MCP server — automate PR quality with LLM-as-Judge, build no-code agents in Atlassian Rovo Studio, build code-based agents with the Copilot SDK and ground them with a simple RAG flow, observe AI workflows and measure token economics/ROI/governance (including a self-hosted SLM demo), and finally demonstrate the complete workflow through the Honeywell Software Engineering Capstone and Adoption Roadmap.

GitHub Copilot Enterprise remains the primary AI engineering platform, but the design now assumes a power-user cohort: core capabilities (chat, inline assistance, code explanation) are recapped briefly, with the majority of Module 2 time spent on advanced, hands-on practice — custom instructions, custom agents, reusable skills, advanced prompting, and agent-mode workflows. Participants then learn two complementary development flavours. In the greenfield flavour, they use GitHub Spec Kit to move from problem framing and requirements through specification, architecture, implementation, testing, review, CI/CD and release. In the brownfield flavour, they learn repository understanding, architecture and dependency analysis, change-impact analysis, preservation of existing design principles and patterns, controlled implementation, regression protection, and app modernization. MCP is treated as a key engineering capability for connecting agents to approved tools, repositories, services and enterprise workflows, and now includes hands-on custom MCP server creation with simple orchestration — the most requested addition from the delivered batch.

A core differentiator of this programme remains the direct comparison between prompt-only development and specification-led development, followed by validation through a proper test strategy. Module 3 recaps practical prompt engineering before moving into context engineering: how to select, structure, minimise and manage repository, design, API, issue, test and specification context. The test strategy module (Module 6) now validates using mocks, stubs and lightweight test doubles, removing infrastructure setup time without weakening the testing method taught.

Agent building now follows a deliberate progression: participants first shape agent behaviour without code in Atlassian Rovo Studio (Module 9), then build an agent in code using the Copilot SDK and extend it with a simple retrieval-grounded (RAG) flow (Module 10) — RAG is taught as a context source for an agent participants have just built, not as a standalone architecture topic. Module 11 combines Agent Prism observability, ROI/token economics, governance, and a self-hosted SLM/LLM demo into a single, minimalistic closing module ahead of the capstone. The programme closes with a working brownfield/greenfield capstone and a measurable adoption roadmap.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.
NIIT ENTERPRISE

Business Problems This Programme Solves

- Slow feature and application implementation cycles caused by repetitive requirements analysis, architecture, coding, test creation, documentation, CI/CD, and review activities.
- Risk of AI-generated changes disrupting brownfield applications, violating existing architecture or design patterns, breaking contracts, or introducing regressions.
- Weak translation of requirements into precise implementation tasks, leading to inconsistent results when teams rely on prompts alone instead of a structured specification and validation workflow.
- Manual and inconsistent testing and PR review, creating review bottlenecks and making it difficult to validate specification compliance, integration behaviour, and regression safety.
- Excessive token consumption and weak context discipline, resulting in avoidable AI cost and lower-quality outputs.
- Limited visibility into AI usage, quality, failure patterns, token economics, and measurable productivity/ROI from AI-assisted engineering.
- Inconsistent baseline understanding of GenAI/LLM concepts across mixed development, testing, and Scrum Master/PO participant groups, previously consuming live session time.
- Limited hands-on exposure to extending AI agents with custom tooling (MCP servers), building agents in code, and grounding agent responses with retrieval (RAG).

Indicative Design and Content Coverage

Pre-requisites

- Participants should have working familiarity with enterprise software engineering environments, SDLC/PDLC practices, Git/GitHub collaboration, and brownfield application development.
- Technical participants should be comfortable with at least one language or stack across Python, .NET Core/C#, ReactJS, Node.js, and relational/database technologies, plus core practices such as branching, pull requests, code review, testing, debugging, and basic design principles/patterns.
- Participants should be prepared to work with an approved GitHub repository and a representative brownfield application. Developers and designers should be able to inspect existing architecture, APIs, dependencies, tests, and coding conventions before making changes.
- Mandatory pre-read/self-paced foundation (to be completed before Day 1): AI fundamentals bootcamp, intro to GenAI/LLM concepts, intro to agents, and responsible AI awareness. This is now a prerequisite gate — not a recommendation — to bring the mixed development/testing/Scrum-Master-PO participant profile to a common baseline and keep Module 1 hands-on-free.
- Access should be arranged for GitHub Copilot Enterprise, GitHub Spec Kit, approved repositories, a sample brownfield codebase, mocking/stubbing test tooling, CI/PR pipelines or mock pipelines, an MCP server starter/orchestration environment, Atlassian Rovo Agent Studio sandbox, Copilot SDK access, Agent Prism sandbox/demo access, and a self-hosted SLM demo environment.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Course: AI Champions Programme for Software Engineering Teams — NIIT Delivery Design

Total Duration: 24 Hours — 3 Full Days (8 hours/day), delivered as an intensive hands-on programme with a logical progression from GenAI/LLM concepts recap and Copilot advanced foundations through prompt/context engineering, SDD, greenfield and brownfield software engineering, testing, MCP and custom MCP server creation, PR quality, no-code and code-based agent building with RAG, observability/economics/governance, and capstone.

Target Roles

- Common foundation for all: Architects, Developers, Testers, DevOps, SRE, Engineering Leads, Product Owners, Designers, and selected Program/TPO/RTE/Scrum roles
- Technical deep-dive emphasis: Architects, Developers, Testers, DevOps, SRE, Engineering Leads
- Design/collaboration emphasis: Product Owners, Designers, TPOs, RTEs, and Scrum roles

Key Outcomes

- Apply an incremental AI-assisted software engineering lifecycle covering GitHub Copilot advanced capabilities, prompt and context engineering, specification, planning, implementation, testing, PR review, MCP-enabled tool integration including custom MCP server creation, no-code and code-based agent building, RAG grounding, observability, economics and adoption.
- Use GitHub Copilot Enterprise at a power-user level: custom instructions, custom agents, reusable skills, advanced prompting, agent-mode workflows, repository-aware assistance, and terminal/CLI and GitHub.com workflows.
- Author structured specifications using GitHub Spec Kit — specification, architecture/plan, tasks, and acceptance criteria — to drive precise, traceable implementation for greenfield features and applications.
- Enhance brownfield applications safely by identifying architecture, dependencies, extension points, design principles, existing patterns, API contracts, tests, and change impact before modifying code, including an app-modernization approach.
- Design and execute a proper test strategy for SDD-developed features — unit, integration, API/database and contract testing — using mocks, stubs and lightweight test doubles.
- Build MCP-enabled engineering workflows that connect agents to approved tools, repositories, APIs and enterprise services, and create a simple custom MCP server with basic orchestration.
- Implement AI-assisted PR quality gates including specification compliance, coding standards, security, test evidence, regression risk, and LLM-as-Judge patterns with human escalation.
- Shape agent behaviour without code using studio-style workflows in Atlassian Rovo Agent Studio, then build an agent in code using the Copilot SDK and ground it with a simple retrieval-augmented (RAG) flow.
- Use Agent Prism to monitor traces, quality signals, failures, token usage and cost patterns; build a compact ROI/token-economics model; and view a self-hosted SLM/LLM demo as a cost and governance lever.
- Integrate the complete workflow into a working capstone use case and an evidence-based 30/60/90-day adoption roadmap.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

Detailed Design

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hrs)	Hands-on	Tools Needs
1. Course Objectives & GenAI/LLM Concepts Recap	Programme objectives and AI Champion role; recap of GenAI, LLM, prompt/context/agent fundamentals (assumes pre-read completed); review of current engineering pain points, success measures, and programme baseline metrics.	**1**	Facilitated discussion (no technical lab): map current-state vs. target-state engineering workflow, confirm baseline measures for implementation cycle time, quality/regression risk, PR review time, testing effort, token usage and AI cost.	Slides, current/target-state workflow canvas, success-metrics worksheet.
2. GitHub Copilot Recap & Advanced Features	Brief recap of core Copilot capabilities (chat, inline assistance, code explanation); advanced hands-on: custom instructions, custom agents, reusable skills, advanced prompting patterns, agent-mode workflows, model/task selection, terminal/CLI and GitHub.com workflows for power users.	**2.5**	Build and apply a custom instruction set and a reusable skill; configure a custom agent; execute an advanced agent-mode task end-to-end and compare against baseline Copilot chat/ inline usage.	GitHub Copilot Enterprise, IDE, GitHub.com, terminal/ CLI, sample repositories, custom instruction/skill templates.
3. Prompt Engineering Recap & Context Engineering	Prompt fundamentals recap: task framing, role/ instruction, constraints, examples, decomposition, iterative prompting, verification. Context engineering: repository context, relevant files/dependencies, architecture/design notes, APIs, standards, issue history, specifications and tests; context selection, layering, compression, task-scoped context, token-aware practices, avoiding context inflation.	**2**	Solve an engineering task with a prompt-only approach, then progressively improve it using structured context. Compare output quality, token usage, iteration count and task completion time.	Copilot, prompt/context templates, repository examples, token usage worksheet.
4. Spec-Driven Development with GitHub Spec Kit / OpenSpec	GitHub Spec Kit introduction and core concepts; SDD fundamentals — requirements, functional/non-functional constraints, interfaces/contracts, acceptance criteria, specification quality; spec → plan → tasks → implementation; prompt-only vs. spec-driven comparison; traceability and change control.	**2**	Create a feature specification using GitHub Spec Kit, generate plan/tasks, implement against the specification, validate acceptance criteria, and compare with a prompt-only baseline.	GitHub Copilot, GitHub Spec Kit, specification templates, acceptance-criteria checklist.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.
NIIT ENTERPRISE

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hrs)	Hands-on	Tools Needs
5. Software Development: SDD to Complete Software SDLC	Use SDD to build a greenfield application/ feature through architecture, project conventions, implementation, test strategy, integration, CI/CD, PR, release readiness Repository archaeology, architecture/ dependency analysis, change-impact analysis, extension points, preservation of existing design principles and patterns, API compatibility, minimal-change strategy, backward compatibility and regression protection; app-modernization patterns and a legacy-to-modern migration demo.	**3.5**	Hands-on: take the specification from Module 4 through implementation, tests, CI/quality checks and PR/release readiness. Validate that the delivered feature matches the specification. Analyse an existing brownfield application, identify affected components and constraints, create a change plan, implement a feature without unnecessary disruption, and verify preservation of existing behaviour;	Copilot Enterprise, GitHub Spec Kit, GitHub repo, CI pipeline, brownfield codebase, architecture/design notes, dependency-analysis tools, change-impact checklist.
6. Test Strategy & Validation	Test strategy derived from SDD, specification-to-test traceability, unit tests, integration tests, API/database validation using mocks and stubs, lightweight test doubles, negative scenarios, contract validation, regression testing, coverage and acceptance-criteria validation.	**1.5**	Derive the test strategy from the SDD feature, create/update unit and integration tests using mocked/stubbed dependencies, inject a deliberate defect, and prove the strategy catches it.	Unit/integration test framework, mocking/ stubbing libraries, API test tools, CI pipeline.
7. MCP-Enabled Workflows & Custom MCP Server Creation	MCP architecture, servers/tools/resources; connecting agents to approved repositories, APIs, databases, documentation and enterprise services; tool selection, permissions, approvals, context/state boundaries, failure handling, security and governance; building a simple custom MCP server with basic orchestration.	**3**	Build an MCP-enabled workflow such as repository analysis → specification validation → result analysis. Then design and stand up a simple custom MCP server exposing one or two tools, wire it into an agent workflow, and document its governance model.	GitHub Copilot/agent environment, MCP server starter template, approved tools/APIs, GitHub assets, workflow canvas.
8. PR Automation, Quality Gates & LLM-as-Judge	AI-assisted PR review, specification compliance, coding/design standards, security checks, test evidence, regression risk, maintainability, evidence-based comments, LLM-as-Judge evaluation rubrics, confidence, human escalation and review-cycle reduction.	**1**	Configure a PR quality flow, run AI review and LLM-as-Judge checks against a deliberately flawed change, capture evidence, compare review time and quality, and close the loop with human approval.	GitHub repo, PR templates, quality checklist, CI pipeline, LLM-as-Judge rubric.
9. Studio-Style No-Code Agents (Rovo Agent Studio)	How non-code/semi-technical users shape agent behaviour through studio-style workflows in Atlassian Rovo Studio: agent authoring for program/product roles, grounding agents on Jira/Confluence sources, triggers and automation rules, actions and connectors, approval flows, human-in-the-loop checkpoints, permission-aware access, and the governance boundary between a studio workflow and an engineering agent.	**2**	Define an agent-assisted workflow in Atlassian Rovo Studio for program reporting, issue triage, or release coordination. Ground it on sample Jira/Confluence data, configure trigger/actions/approval steps, test against representative data, and hand it over to technical owners.	Atlassian Rovo Studio, Jira/Confluence sandbox tenant, sample project and space, workflow canvas, business-user templates.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.
NIIT ENTERPRISE

Module Name	Sub-topics / Detailed Technical Coverage	Duration (Hrs)	Hands-on	Tools Needs
10. Code-Based Agent Development (Copilot SDK) & Simple RAG Flow	Building an agent in code using the Copilot SDK — agent structure, tool wiring, execution loop; extending the agent with a simple retrieval-augmented (RAG) flow — indexing a small document set, a retrieval step, and grounding agent responses. RAG is taught as a context source for the agent just built, not as a standalone architecture topic.	**2**	Build a minimal code-based agent using the Copilot SDK; add a simple retrieval step to ground its responses against a small sample knowledge set; compare grounded vs. ungrounded outputs.	Copilot SDK, code editor/IDE, sample document set for retrieval, agent starter template.
11. Observability, ROI & Governance	Agent Prism — traces, replay, failure patterns, quality/control signals, token usage, cost-leak visibility; engineering productivity vs. production impact, cost per accepted production-quality output, token governance, business-value measurement, governance and leadership reporting; self-hosted SLM/LLM demo as a cost/control lever.	**1.5**	Inspect sample Agent Prism traces and failures, identify token/cost leakage; calculate before/after metrics and build a compact ROI estimate; observe a self-hosted SLM demo and discuss when an AI workflow should be scaled, tuned, restricted or retired.	Agent Prism sandbox/demo, sample run telemetry, ROI worksheet, token-economics template, self-hosted SLM demo environment.
12. Honeywell Capstone & Adoption Roadmap	Teams integrate the complete engineering journey into one realistic adoption plan: GitHub Copilot Enterprise workflow, reusable skill/spec pack, greenfield SDD or brownfield/modernization approach, test strategy and evidence, custom MCP server/workflow, PR quality automation, LLM-as-Judge, no-code and code-based agent approach with RAG grounding, Agent Prism monitoring model, and ROI model. Final review includes business case, governance controls, and enterprise rollout roadmap.	**2**	Capstone: present one software engineering use case with solution blueprint, specification/plan, implementation approach, test strategy and evidence, MCP workflow, PR/LLM-Judge evidence, agent approach (no-code and/or code-based with RAG), Agent Prism monitoring design, ROI model, and 90-day adoption roadmap.	Greenfield starter or brownfield application, GitHub repo/PR pipeline, GitHub Spec Kit, MCP tools, Agent Prism, capstone canvas, evaluation rubric.
Total		**24**		

Coverage Alignment to the Uploaded Honeywell Training Sheet

- The programme is structured as a 12-module progression so each concept builds on the previous one: GenAI/LLM concepts recap → GitHub Copilot advanced features → prompt engineering recap and context engineering → SDD greenfield (Spec Kit) → brownfield engineering and app modernization → mock/stub-based test strategy → MCP and custom MCP server creation → PR quality/LLM-as-Judge → no-code agents (Rovo Agent Studio) → code-based agents (Copilot SDK) and simple RAG grounding → observability/ROI/governance and self-hosted SLM demo → Honeywell Software Engineering Capstone and Adoption Roadmap.
- Discovery and Launch themes are reflected through the GenAI/LLM baseline recap, requirements framing, greenfield application planning, brownfield repository understanding, context engineering, structured specifications using GitHub Spec Kit, use-case identification, and translating requirements into implementation-ready work.
- Development and Test themes are strengthened through GitHub Copilot agent mode and advanced features, SDD, complete SDLC execution, repository/context management, AI-assisted implementation, design-principle compliance, mock/stub-based integration testing, PR automation, code review, LLM-as-Judge, reusable skills, MCP/tool integration including custom server creation, and AI usage cost optimisation.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.

- COTS AI tool themes are extended through MCP server and tool-integration patterns, now including hands-on custom MCP server creation. RAG is now included as a light, applied topic — taught as a context-grounding technique for the code-based agent built in Module 10 — reflecting direct participant and stakeholder demand from the delivered batch.
- Agent-building coverage now spans both the no-code (Atlassian Rovo Agent Studio) and code-based (Copilot SDK) approaches, giving business/product and engineering roles a shared, progressive vocabulary for how agents are authored and governed.
- Agent Prism, ROI/token economics, and governance remain the practical enterprise monitoring layer, now combined into a single closing module alongside a self-hosted SLM/LLM demo so teams connect AI-assisted engineering activity directly to quality, cost, adoption, and ROI.

Learning Pedagogy

- Instructor-led sessions use two realistic software engineering journeys: build a greenfield application feature using SDD, and manage a change (including modernization) in an existing brownfield application without disrupting established behaviour. A mandatory pre-read establishes the GenAI/LLM baseline before Day 1, so Module 1 opens with a concept recap and discussion rather than a lab, and GitHub Copilot advanced capabilities are introduced early so participants have a strong practical tool foundation before applying the engineering methods.
- Guided labs follow a complete engineering lifecycle: recap concepts, use Copilot at an advanced level, establish context, specify with GitHub Spec Kit, plan, implement, test with mocks/stubs, review, integrate tools through MCP (including building a custom MCP server), observe AI behaviour, measure economics, and demonstrate adoption. The brownfield path adds repository archaeology, impact analysis, regression protection, and an app-modernization walkthrough.
- Participants recap prompt engineering and then compare prompt-only implementation with structured context and SDD workflows, evaluating differences in implementation time, output quality, context usage, testing effort, review effort, specification compliance, token usage and cost.
- Design-thinking activities help participants frame the problem, identify constraints, understand existing architecture, and define acceptance criteria before asking AI to implement a solution.
- Testing and PR quality clinics connect AI productivity directly to confidence: test strategy from the specification, unit/integration testing validated with mocks and stubs (no Testcontainers dependency), regression checks, specification compliance, and LLM-as-Judge review.
- MCP is taught as a practical mechanism for extending AI-assisted engineering with governed tools and enterprise context, culminating in participants building their own simple custom MCP server rather than only consuming pre-built ones.
- Agent-building labs follow a deliberate no-code-to-code progression: participants first shape agent behaviour without code in Atlassian Rovo Agent Studio, then build an agent in code using the Copilot SDK and ground it with a simple RAG flow — introducing retrieval as a natural extension of an agent they have just built, rather than as an abstract standalone topic.
- Agent Prism labs connect traces, token usage, cost, quality and failure patterns to the programme success metrics. The combined observability/ROI/governance module — including a self-hosted SLM/LLM demo — converts these observations into measurable business outcomes before the final capstone.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.
NIIT ENTERPRISE

Free Value-Adds / Client Handholding

- 30-day catch-up review session at no additional cost to review what participants have applied, what has been deployed, and where adoption is blocked.
- One complimentary masterclass after the 30-day review focused on advanced lessons learned, common implementation mistakes, SDD maturity, testing strategies, and next-step AI engineering patterns.
- Post-training assessment framework to review feature implementation cycle time, PR review time, regression/ defect signals, reusable skill/spec adoption, token spend discipline, and early ROI signals from deployed or piloted workflows.
- Short gap-closure recommendations based on participant progress, leadership expectations, and identified blockers in governance, tooling, brownfield adoption, testing, MCP server ownership, or agent-workflow design.

Delivery Recommendations

- Recommended audience: Honeywell software engineering teams with role-based participation from architects, developers, testers, DevOps, SRE, engineering leads, designers, and selected product/program roles.
- Preferred delivery model: blended delivery — overview sessions online, use-case identification/design-thinking workshops interactive, and hands-on brownfield/MCP/agent-building working sessions in person for shortlisted use cases.
- Primary tool/platform: GitHub Copilot Enterprise (advanced features) for core AI-assisted development workflows, GitHub Spec Kit for SDD, mocking/stubbing frameworks for integrated testing , GitHub PR/CI workflows for quality gates, MCP servers/tools (including custom server creation) for governed engineering integrations, Atlassian Rovo Agent Studio for no-code agent authoring, the Copilot SDK for code-based agent development with a simple RAG flow, a self-hosted SLM/LLM demo environment, and Agent Prism for monitoring, observability, token economics and ROI.
- Recommended batch design: mandatory GenAI/LLM pre-read, followed by a common foundation and two targeted engineering journeys — greenfield SDD/complete SDLC and brownfield enhancement/app modernization — with Copilot advanced features and prompt/context engineering established first, MCP (including custom server creation) integrated after the core development workflow is understood, and agent building progressing from no-code (Rovo Agent Studio) to code-based (Copilot SDK) with RAG grounding.
- Outcome expectation: every batch should leave with a working greenfield SDD workflow, a working brownfield feature-change/modernization approach, a documented mock/stub-based test strategy, a custom MCP server and reusable engineering assets, PR quality automation, a no-code agent workflow and a code-based RAG-grounded agent, Agent Prism observability evidence, a token/cost and ROI model, and a completed Honeywell Software Engineering Capstone and Adoption Roadmap.
- Client enablement needed: approved GitHub access, a representative greenfield application starter and brownfield full-stack repository, architecture/design documentation, coding standards, existing tests, mocking/stubbing test tooling, approved MCP server starter environment and orchestration sandbox, Atlassian Rovo Agent Studio sandbox tenant, Copilot SDK access, a self-hosted SLM/LLM demo environment, PR/CI pipelines or mock pipelines, reusable standards/checklists, and Agent Prism practice environment or demo dataset.

© NIIT, All Information within this document is Intellectual property of NIIT Ltd.
No part of this document can be shared or used without the permission of NIIT Ltd.